

Administration Distante et PowerShell Remoting

Ce document détaille les mécanismes d'exécution de code à distance via PowerShell. L'objectif est de s'affranchir des connexions RDP individuelles pour administrer des parcs entiers de serveurs depuis une console d'administration centralisée, de manière sécurisée et parallélisée.

1. Architecture et Prérequis WinRM

Le PowerShell Remoting repose sur **WinRM** (Windows Remote Management), l'implémentation Microsoft du standard WS-Management (basé sur SOAP/HTTP).

- **Protocole et Ports** : WinRM écoute par défaut sur les ports **5985** (HTTP) et **5986** (HTTPS). Bien que qualifié de "HTTP", le trafic est nativement chiffré au niveau de la couche réseau dès lors que les machines sont dans un domaine Active Directory (via le protocole Kerberos).
- **Activation du point d'écoute** : Sur un serveur Windows (à partir de Windows Server 2012 R2), WinRM est actif par défaut. S'il ne l'est pas, ou pour configurer un poste client d'administration, la commande de provisionnement complet est :

Enable-PSRemoting -Force

Cette commande démarre le service WinRM en automatique, crée le point d'écoute (Listener) et configure les règles du pare-feu Windows.

2. Session Interactive (One-to-One)

Pour l'investigation ou le dépannage ciblé d'un serveur unique, PowerShell permet d'ouvrir une session interactive (équivalent fonctionnel d'une connexion SSH sous Linux).

- **Entrer dans une session distante** :

Enter-PSSession -ComputerName srv-01

Le prompt de la console change (ex: [srv-01]: PS C:\>) pour indiquer que toutes les commandes saisies sont désormais exécutées directement sur le processeur du serveur distant.

- **Spécifier des identifiants (si compte différent)** :

Enter-PSSession -ComputerName srv-02 -Credential (Get-Credential)

- **Clôturer la session proprement** :

(Note : Fermer la fenêtre de la console coupe également la session active en arrière-plan).

3. Exécution Distribuée (One-to-Many) : Invoke-Command

C'est le cœur de l'administration de parc. Contrairement à une boucle foreach contenant un Enter-PSSession (qui exécuterait les actions séquentiellement), Invoke-Command envoie la charge de travail aux serveurs distants de manière **simultanée et parallélisée**.

- **Syntaxe de base :**

```
Invoke-Command -ComputerName srv-01, srv-02 -ScriptBlock {  
    Get-Service -Name wuauserv | Restart-Service -PassThru  
}
```

- **Comprendre la restitution d'objets (Sérialisation) :**

Les objets renvoyés par Invoke-Command sont sérialisés (convertis en texte XML structuré) pour traverser le réseau, puis désérialisés côté client. Ils perdent leurs méthodes actives (on ne peut plus faire un .Stop()), mais conservent l'intégralité de leurs propriétés de données.

PowerShell ajoute automatiquement une propriété **PSComputerName** à chaque objet retourné, permettant d'identifier immédiatement quel serveur a généré la donnée.

La portée des variables et le modificateur \$Using:

C'est l'erreur la plus fréquente en production. Le bloc de script (-ScriptBlock) s'exécute dans un espace mémoire isolé sur le serveur distant. Il ne connaît absolument pas les variables définies sur votre poste client local.

- **Approche en échec :**

```
$ServiceCible = "Spooler"  
# ERREUR : $ServiceCible est vide et inconnue sur srv-01  
Invoke-Command -ComputerName srv-01 -ScriptBlock { Get-Service -Name $ServiceCible }
```

- **Approche valide (Le modificateur \$Using:) :**

```
$ServiceCible = "Spooler"  
Invoke-Command -ComputerName srv-01 -ScriptBlock { Get-Service -Name  
$Using:ServiceCible }
```

Le mot-clé `$Using`: demande à PowerShell d'injecter la valeur de la variable locale dans le script distant avant son exécution.

4. Sessions Persistantes (PSSession)

Lors de l'utilisation répétée de `Invoke-Command` vers les mêmes serveurs au cours d'un script, la négociation WinRM (authentification, création du canal) consomme du temps. La création de sessions persistantes optimise drastiquement les performances.

- **Création d'une variable de sessions (Pool) :**

```
$Sessions = New-PSSession -ComputerName srv-01, srv-02
```

- **Exécution au travers des sessions ouvertes :**

```
Invoke-Command -Session $Sessions -ScriptBlock { Get-Process | Measure-Object }
```

- **Destruction des sessions (libération des ressources réseau et RAM) :**

```
$Sessions | Remove-PSSession
```

5. Exécution Asynchrone (PowerShell Jobs)

Lorsqu'un bloc de script prend plusieurs minutes (ex: recherche récursive de fichiers, installation lourde de rôles Windows), il fige la console d'administration. L'exécution asynchrone permet de rendre la main immédiatement à l'opérateur.

- **Lancement d'une tâche d'arrière-plan distante :**

```
Invoke-Command -ComputerName srv-01, srv-02 -ScriptBlock { Start-Sleep -Seconds 30;  
Get-Date } -AsJob
```

- **Gestion du cycle de vie des tâches :**

- `Get-Job` : Affiche l'état des tâches en mémoire (Running, Completed, Failed).
- `Receive-Job -Id 1 -Keep` : Récupère les résultats d'une tâche terminée (le `-Keep` empêche la suppression des données après la première lecture).
- `Remove-Job -Id 1` : Supprime définitivement la tâche et libère la mémoire.

6. Contraintes de Sécurité Avancées en Entreprise

A. Les environnements hors-domaine (TrustedHosts)

Si le poste d'administration et les serveurs ne sont pas dans le même domaine Active Directory (ex: poste de travail en Workgroup et serveurs en DMZ), Kerberos ne peut pas authentifier la

cible. Il faut déclarer manuellement que le client fait confiance au serveur distant pour forcer une authentification NTLM ou Basic :

```
Set-Item WSMan:\localhost\Client\TrustedHosts -Value "srv-01, 10.100.101.*" -Force
```

B. La problématique du "Double-Hop" (Double Rebond)

Lorsque vous êtes connecté en WinRM de *Poste-Client* vers *srv-01*, et que vous demandez à *srv-01* d'accéder à un partage réseau ou un service situé sur *srv-02*, l'accès est immédiatement refusé par le système.

- **Cause cryptographique :** Pour éviter le vol d'identité, Kerberos interdit par défaut à un serveur de relayer vos identifiants à une troisième machine (le 2ème rebond). *srv-01* se présente à *srv-02* en tant que "Connexion Anonyme" (Anonymous Logon).
- **Solutions d'architecture :**
 1. Passer les identifiants explicitement dans le script exécuté sur *srv-01* via un objet [PSCredential].
 2. Activer la délégation **CredSSP** (Fortement déconseillé en production aujourd'hui pour des raisons de sécurité liées aux attaques de type *Pass-The-Hash*).
 3. Utiliser la **Délégation Contrainte Basée sur les Ressources (RBCD)** native dans Active Directory (Standard actuel recommandé).